

Week 3: Object-Oriented Programming

Valid planning-domain objects, explicit state ownership, and collaboration

School of Mechanical Engineering, Soongsil University
2026-2 Applied Programming

Week 2 represented a path as a numerical array and verified functions that measured and sampled it. That representation is effective for numerical calculation, but a larger program must also answer questions that an array cannot answer by itself: Which row is the start configuration? Which path belongs to which planning problem? Is a failure result allowed to contain a path? Who is allowed to append an execution result, and which object owns that changing history?

Object-oriented programming gives these roles names and places related data and operations behind explicit boundaries. The purpose is not to replace every function with a class. The supplied numerical function remains a pure function. Classes organize domain meaning, preserve valid state, and make collaboration between components visible and testable.

This chapter develops one continuous example. A sequence of numerical configurations becomes a `Path`. A `PlanningProblem` composes a start, goal, and bounds. A `PlanResult` represents either a valid path or an explained failure. A mutable `PlanningRun` records compatible results. The final section contains Assignment 3, Part A and the complete Week 3 repository and LMS submission procedure.

After completing this chapter, a student should be able to

- distinguish a class, instance, attribute, method, and the conventional `self` parameter in an executed Python example.
- use `@dataclass` while explaining which methods it generates and which validation it does not perform.
- state and enforce constructor invariants in `__post_init__`.
- distinguish frozen value objects from one explicitly mutable state owner and avoid shared mutable defaults.
- choose inheritance for an actual is-a compatibility relationship and composition for the has-a relationships in this domain model.
- test valid construction, invalid construction, successful updates, and rejected updates that must leave state unchanged.
- complete, test, package, and submit the Week 3 work that Week 4 will extend at the JSON boundary.

The package does not generate a path. It does not check collisions, control a robot, parse JSON, or run a physics simulation. Those limitations are part of the contract, not missing claims.

1 Numerical data need domain meaning

1.1 The same numbers can play different roles

Consider the supplied Week 2 path:

Code excerpt - supplied coordinates

```
import numpy as np

path_xy = np.array([
    [0.0, 0.0],
    [1.0, 1.4],
    [2.4, 1.1],
    [3.2, 2.0],
], dtype=np.float64)
```

This excerpt creates `path_xy`. It does not print a result. Its shape `(4, 2)` states that four rows each contain two coordinates. The array supports interpolation and length calculations, but the type alone does not identify the first row as the required start, the final row as the required goal, or the rectangle within which every waypoint must remain. A second array with the same shape might be sensor samples, obstacle points, or a completely unrelated table.

A **domain object** gives a concept in the problem domain a name, stores the data that define it, and exposes operations that belong to its responsibility. For Week 3, the domain is a small planning model. A **configuration** is one point in the state space used by the planning problem. The classroom example uses two coordinates so that it can be drawn as a point-robot path. The object contract permits other positive dimensions because a later robot configuration may contain several joint coordinates. A tool-center point does not, by itself, specify every joint of a robot arm. Here the two coordinates, measured in metres, describe a planar point robot. The lesson is the software representation, not the conversion from tool coordinates to joint angles.

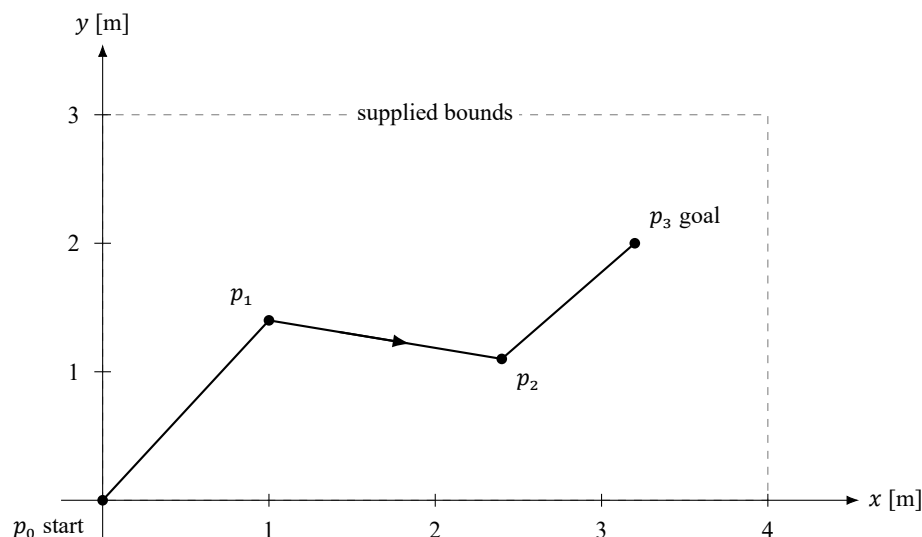


Figure 3.1: The supplied path is the ordered polyline p_0, p_1, p_2, p_3 . The dashed rectangle marks bounds $[0, 4] \times [0, 3]$ m. It is not an obstacle map.

The line in Figure 3.1 does not describe a planning algorithm. It displays supplied coordinates in order. Each point will become a **Configuration**, and their ordered collection will become a **Path**. Week 3 asks whether the program can construct a coherent set of objects around that supplied path and reject an incoherent set before it reaches later code.

1.2 Responsibility is narrower than “everything related”

An object should not absorb every operation that mentions its data. The `Path` object owns an ordered sequence of configurations and can expose its start, goal, array representation, and length. The numerical summation for the length is still performed by the supplied pure function `polyline_length`. This boundary gives two useful properties:

- domain code states what a path means and what must remain valid.
- numerical code performs one calculation without owning changing planning state.

Object-oriented programming and functional decomposition therefore cooperate. The design question is not “functions or classes?” but “which component should own this data, this rule, and this change?”

2 A class defines a kind of object

The first examples are small programs, not the unfinished Assignment package. Open **Anaconda Prompt**, activate the existing course environment, and make a practice folder outside any assignment repository. In the commands below, the folder is placed in your home directory:

Run - Anaconda Prompt, create the practice folder

```
conda activate applied-programming-2026
cd /d "%USERPROFILE%"
mkdir week03_practice
cd week03_practice
notepad 01_instances.py
```

If the folder already exists, skip `mkdir`. Save the program below as `01_instances.py`. Its execution command and output are paired below. Later examples specify a new filename when the class definition changes. They do not silently modify the class in an earlier file. On macOS or Linux, use the activated terminal, replace the home-directory command with `cd ~`, and use your text editor. The Python programs themselves are unchanged.

Activation selects the course Python and installed packages. The two `cd` commands change the working folder, `mkdir` creates the practice folder, and `notepad` opens the source file for editing. Running `python 01_instances.py` executes the saved file, not unsaved editor contents. Type shell commands in Anaconda Prompt and write Python code in the file. The program does not ask you to type coordinates while it runs. Its input data are the tuples already written in the source. Its `print` calls display the output shown below. These practice programs do not create submission files.

2.1 Class, instance, attribute, and method

A **function** is a piece of code that performs a task when called. An **argument** is an input value supplied by the calling code. A call supplies any arguments and runs the function body. The code making that call is the **caller**. Caller describes a role, not a different kind of Python object. It may be another function or code at the top level of a script. A **return** statement sends a value back to that calling code. It does not display the value. `print` displays text in the execution window.

A **class** defines a new object type. An **instance** is one object created from that class. An **attribute** is a named value reached through the object. A **method** is a function associated with the class and called through an object. The following class illustrates these relationships before adding the full Assignment input contract:

Input

The tuples `(0.0, 0.0)` and `(3.2, 2.0)` are written in the calling code. The program creates two instances from these values without asking for keyboard input.

Code - 01_instances.py

```
class Configuration:
    def __init__(self, values):
        self.values = tuple(values)

    def dimension(self):
        return len(self.values)

q_start = Configuration((0.0, 0.0))
q_goal = Configuration((3.2, 2.0))

print(q_start.values)
print(q_start.dimension())
print(q_start is q_goal)
```

Run - Anaconda Prompt, week03_practice

```
python 01_instances.py
```

Output

```
(0.0, 0.0)
2
False
```

`Configuration` is the class. `q_start` and `q_goal` are two instances. The expression `q_start.values` reads an attribute. The expression `q_start.dimension()` calls a method. The final result is `False` because the two names refer to different instance objects. In `q_start = Configuration((0.0, 0.0))`, the class call receives the tuple and creates an instance. Python passes the new instance as `self` and the tuple as `values` to `__init__`. The initializer stores `self.values`. After initialization succeeds, `q_start` refers to the instance. The initializer itself does not return the instance. Assigning this attribute does not change `q_goal`.

For the second printed line, `q_start.dimension()` returns the integer 2 to the surrounding code, and `print` displays it. The method contains no printing operation. Replacing that line with `count = q_start.dimension()` would save 2 in `count` without displaying a line. This distinction between a returned value and printed output will also apply to the larger package.

2.2 What `self` means during a method call

In the method definition, `self` is the conventional name for the instance on which the method operates. Python supplies that instance when the method is called through an object. These two calls are equivalent:

Code excerpt - two equivalent method calls

```
q_start.dimension()
Configuration.dimension(q_start)
```

The first form is normal application code. The second form exposes the call mechanism: `q_start` becomes the

first argument. The name `self` is not a keyword, but following the convention makes Python code readable to other programmers.

Both expressions return 2. In a saved script, a bare expression does not display its return value. To inspect either call, put it inside `print(...)` in the same `01_instances.py` file after `q_start` has been created.

Example 3.1 Two instances must not accidentally share changing state

An instance attribute belongs to one instance. A class attribute belongs to the class and is therefore visible through every instance unless an instance overrides it. A mutable class attribute can create an unintended shared list. Save this independent example as `shared_history.py` in the practice folder and run `python shared_history.py`:

Code - `shared_history.py`

```
class BadPlanningRun:
    results = []

first = BadPlanningRun()
second = BadPlanningRun()
first.results.append("failed")

print(second.results)
print(first.results is second.results)
```

Run - Anaconda Prompt, `week03_practice`

```
python shared_history.py
```

Output

```
['failed']
True
```

The `append` call receives the string `"failed"`, changes the existing list, and returns `None`. It does not print the new list. The following `print(second.results)` exposes the change. Although the update was made through `first`, `second` observes it because both lookups reach the same class-level list. A result history is not shared course configuration. It belongs to one planning run. The later `default_factory` example will create a distinct list for every instance.

2.3 Instantiation should establish a usable initial state

Calling a class object creates an instance. When the class defines `__init__`, Python calls it to initialize the new instance. A constructor is therefore an important public boundary: downstream code should not have to ask whether an object was only half initialized.

The explicit class above is still incomplete. It accepts `Configuration()`, `Configuration((True, False))`, and `Configuration(1.0, float("nan"))`. Each value is legal Python, yet none is a valid configuration under this package contract. A class organizes syntax. The program must still implement engineering validity rules.

3 Data classes reduce boilerplate, not responsibility

3.1 Generated methods come from annotated fields

An annotation such as `values: tuple[float, ...]` says that the intended field value is a tuple containing floating-point numbers. The ellipsis permits different tuple lengths. The `@dataclass` decorator inspects these annotated fields and can generate methods such as `__init__`, `__repr__`, and `__eq__`. Save this independent example as `dataclass_basics.py`. It replaces the handwritten initializer in the previous example rather than extending it. Save it in `week03_practice` and run `python dataclass_basics.py`:

Code - `dataclass_basics.py`

```
from dataclasses import dataclass

@dataclass
class Configuration:
    values: tuple[float, ...]

q1 = Configuration((0.0, 0.0))
q2 = Configuration((0.0, 0.0))

print(q1)
print(q1 == q2)
```

Run - Anaconda Prompt, `week03_practice`

```
python dataclass_basics.py
```

Output

```
Configuration(values=(0.0, 0.0))
True
```

Both constructor calls receive `(0.0, 0.0)`. They create separate instances. `print(q1)` uses the generated representation, and `q1 == q2` compares their field values to produce `True`. No handwritten initializer, representation or equality method is needed here. Field order follows the annotations in the class definition.

The annotation does not generally check runtime values. Add the next two lines to the end of that file, save, and run it again. The earlier two output lines remain, followed by the new line below. The call still constructs an instance unless the program adds validation:

Code - append to `dataclass_basics.py`

```
bad = Configuration(values="not numerical coordinates")
print(bad)
```

Run - Anaconda Prompt, week03_practice

```
python dataclass_basics.py
```

Output (additional line)

```
Configuration(values='not numerical coordinates')
```

A type hint communicates an intended interface to readers and tools. It is not a replacement for runtime validation at a public engineering boundary.

3.2 An invariant belongs to every valid state

An **invariant** is a condition that must hold for every successfully constructed public object and remain true after every successful public update. The Week 3 `Configuration` invariant is:

- values form one non-empty dimension.
- every element is numerical, real, non-Boolean, and finite.
- the instance owns a normalized tuple of Python `float` values.

These clauses answer different questions. `Shape` validates representation. Real, finite values validate numerical usability. Rejecting Boolean values prevents `True` and `False` from being interpreted as coordinates merely because Python's Boolean type is related to integers. Tuple normalization establishes ownership.

3.3 `__post_init__` checks generated initialization

The generated data-class initializer assigns the annotated fields and then calls `__post_init__` when it is present. A manually written `__init__` does not gain that automatic call. This next complete practice program adds just one validity rule: the tuple must not be empty. Save it as `02_dataclass_property.py` in `week03_practice`. The complete Assignment class will also need the numerical checks stated above.

Read this file as construction followed by observation. Its `main` function supplies the input list and prints each observation. `@property` lets `q.dimension` compute the coordinate count without parentheses. `frozen=True` rejects later field assignment. This example uses `object.__setattr__` only during initialization to store the normalized tuple. The final `try/except` blocks deliberately attempt invalid operations and print the caught errors, allowing the program to continue. The following sections examine these mechanisms in detail.

Input

The calling code starts with `[0.0, 0.0]`, then changes that list and attempts two operations that should be rejected. All values are supplied in the file, not typed while it runs.

Code - 02_dataclass_property.py, definition

```

from dataclasses import FrozenInstanceError, dataclass

@dataclass(frozen=True)
class Configuration:
    values: tuple

    def __post_init__(self):
        owned = tuple(self.values)
        if not owned:
            raise ValueError("values must not be empty")
        object.__setattr__(self, "values", owned)

    @property
    def dimension(self):
        return len(self.values)

```

The initializer first establishes the stored tuple. The calling code below then checks that later changes to the original list do not replace its coordinates. Append it to the same file, below the class definition.

Code - 02_dataclass_property.py, calling code

```

def main():
    source = [0.0, 0.0]
    q = Configuration(source)
    source[0] = 99.0

    print("caller list:", source)
    print("stored tuple:", q.values)
    print("q.dimension:", q.dimension)
    print("equal values:", q == Configuration((0.0, 0.0)))

    try:
        Configuration(())
    except ValueError as error:
        print("empty input rejected:", error)

    try:
        q.values = (9.0, 9.0)
    except FrozenInstanceError:
        print("field reassignment rejected: FrozenInstanceError")

if __name__ == "__main__":
    main()

```

Save the file and run it from week03_practice:

Run - Anaconda Prompt, week03_practice

```
python 02_dataclass_property.py
```

Output

```
caller list: [99.0, 0.0]
stored tuple: (0.0, 0.0)
q.dimension: 2
equal values: True
empty input rejected: values must not be empty
field reassignment rejected: FrozenInstanceError
```

The input list originally contains `[0.0, 0.0]`. After construction, changing its first element to `99.0` changes the first printed line but not the stored tuple. The two coordinates give dimension 2, and another object with the same coordinates compares equal. The empty input raises `ValueError` instead of producing a usable configuration. The matching handler prints the fifth line, then execution continues to the forbidden field assignment. These two rejection lines are ordinary printed messages from exception handlers, not unhandled tracebacks. They are expected observations, not defects to repair in this practice file.

3.4 A property reads like an attribute but runs a getter

Earlier, `q_start.dimension()` explicitly called a method. In this new definition, `@property` makes `q.dimension` call the getter without parentheses. Both compute a length. The public spelling has changed, so the two definitions must not be mixed. From this point onward the Assignment interface uses `q.dimension`, while `q.as_array()` and `path.length()` remain method calls.

No setter is supplied for `dimension`, so callers cannot assign a new dimension through this property. That does not make every object returned by a property immutable. A getter that returned a list would still expose that list. The next section addresses who owns such mutable data.

4 State ownership must be visible in the design

4.1 Frozen fields and owned values solve different problems

The last practice definition added `frozen=True`. The package uses the same option for configuration, bounds, path, problem, and result objects. The last `try` block in the program attempts field assignment and catches `FrozenInstanceError`. The essential operation is:

Code excerpt - the rejected assignment

```
q = Configuration((0.0, 0.0))
q.values = (9.0, 9.0) # FrozenInstanceError
```

This behavior emulates read-only fields. It is not deep immutability. If a frozen field stores a mutable list or a caller-owned NumPy array, that nested object can still change. The following design is therefore unsafe even though the data class is frozen:

Code - array_aliasing.py

```

from dataclasses import dataclass
import numpy as np

@dataclass(frozen=True)
class UnsafeConfiguration:
    values: np.ndarray

source = np.array([0.0, 0.0])
q = UnsafeConfiguration(source)
source[0] = 99.0
print(q.values)

```

Run - Anaconda Prompt, week03_practice

```
python array_aliasing.py
```

Output

```
[99.  0.]
```

Save the complete array example above as `array_aliasing.py` and run `python array_aliasing.py`. The field was never reassigned. The array object stored in the field was mutated through another name. This is **aliasing**: multiple references lead to the same mutable object.

The practice class has shown the mechanism. The Assignment class must apply it to a stronger public contract. Its `ConfigurationLike` annotation names the allowed sequence or array input. Its internal `_normalized_values` helper collects the repeated representation and numerical checks. This helper is implemented in TODO 1, not supplied by Python. It must reject a Boolean before combining mixed values into an array, because `[True, 1.0]` can otherwise become two floating-point numbers. Conversion errors from a ragged sequence must also be reported as `PlanningModelError`, the package's public invalid-input exception.

The next block is an *Assignment implementation excerpt*, not a standalone practice program. The full module supplies its imports and other definitions. It validates the input and normalizes it to an owned tuple inside `__post_init__`. Because the data class is already frozen while post-initialization runs, it uses `object.__setattr__` for that one controlled normalization:

Code excerpt - Assignment Configuration

```

@dataclass(frozen=True)
class Configuration:
    values: ConfigurationLike

    def __post_init__(self) -> None:
        owned = _normalized_values(self.values, name="values")
        object.__setattr__(self, "values", owned)

```

This is not permission for ordinary callers to change the object. It is an implementation technique used while the constructor establishes the final valid representation.

4.2 Returning a copy protects the object's internal representation

Numerical callers still need a NumPy array. This next method belongs to the same `Assignment Configuration` class. It returns a new array each time:

Code excerpt - the same class, `as_array`

```
def as_array(self) -> NDArray[np.float64]:
    return np.array(self.values, dtype=np.float64, copy=True)
```

Read the caller below as a preview. Return to run it after the Assignment installation procedure and TODO 1–2 are complete. Save the caller as `array_copy.py` in `week03_practice`, then run `python array_copy.py` there. Without installation, the import fails with `ModuleNotFoundError`. An installed but unfinished package instead reaches `NotImplementedError` at a TODO.

Code - `array_copy.py`, after TODO 1–2

```
from ap_week03_planning import Configuration

q = Configuration((1.0, 2.0))
exposed = q.as_array()
exposed[0] = 100.0

print(exposed)
print(q.values)
```

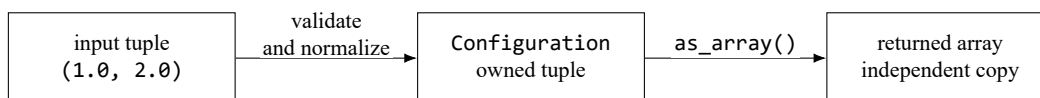
Run - Anaconda Prompt, `week03_practice`

```
python array_copy.py
```

Output

```
[100.    2.]
(1.0, 2.0)
```

The constructor receives `(1.0, 2.0)`. The method returns a separate array to `exposed`. Changing its first element to `100.0` changes the first output line, while the stored tuple on the second line remains `(1.0, 2.0)`. This is an ownership guarantee. It costs an allocation, so a performance-critical system would measure the trade-off. Week 3 prioritizes a small and unambiguous public contract.



Changing the returned array does not change the stored tuple.

Figure 3.2: The input tuple becomes the object's owned values. The caller receives a separate array from `as_array()` and can change it independently.

4.3 One mutable object owns result history

Not all state should be frozen. A planning run accumulates attempts in order. Before composing the full planning model, use a smaller history that stores text labels. Save the following complete program as `practice_history.py` in `week03_practice`, then run `python practice_history.py`. This class is a teaching example, not the Assignment implementation.

Input

The caller below supplies "first attempt" and a blank string. These are text labels used to expose ownership and rejection, not the richer `PlanResult` objects required in the Assignment.

Code - `practice_history.py`, definition

```
from dataclasses import dataclass, field

@dataclass
class PracticeHistory:
    _labels: list[str] = field(default_factory=list, init=False, repr=False)

    def record(self, label: str) -> None:
        if not isinstance(label, str) or not label.strip():
            raise ValueError("label must be a non-empty string")
        self._labels.append(label)

    @property
    def results(self) -> tuple[str, ...]:
        return tuple(self._labels)
```

Continue in the same file with this caller. The final two lines are the script entry-point convention used in Week 1. Running the file calls `main`, whereas importing its class for tests does not run the demonstration.

Code - `practice_history.py`, calling code

```
def main():
    first = PracticeHistory()
    second = PracticeHistory()
    earlier_snapshot = first.results

    first.record("first attempt")
    print("first history:", first.results)
    print("second history:", second.results)
    print("earlier snapshot:", earlier_snapshot)

    before = first.results
    try:
        first.record(" ")
    except ValueError as error:
        print("invalid update rejected:", error)

    print("history unchanged:", first.results == before)

if __name__ == "__main__":
    main()
```

After saving both parts in the same file, run it from the practice folder:

Run - Anaconda Prompt, week03_practice

```
python practice_history.py
```

The output makes the separate histories and rejected update visible:

Output

```
first history: ('first attempt',)
second history: ()
earlier snapshot: ()
invalid update rejected: label must be a non-empty string
history unchanged: True
```

`first.record("first attempt")` receives a label and returns `None` after appending it only to `first`'s internal list. Reading `first.results` returns a tuple snapshot. The later `print` displays that snapshot. The empty tuple on the second line shows that `second` owns another history. The third line is the snapshot taken before the append, so it also remains empty. The blank-string input is rejected, and the last `True` confirms that the history after rejection equals the history saved before the call. The stored history, the returned snapshot and its printed representation are related but distinct.

`default_factory=list` supplies the function `list` for the generated initializer to call for each new instance. It does not supply one already-created `list()` object to share. `init=False` keeps the internal list out of the public constructor arguments. `repr=False` omits it from the generated textual representation. Neither option makes the list private by itself.

The leading underscore says direct external use is not part of the public interface. The `results` property returns a tuple snapshot. Appending a later result creates a new history state, but does not change a previously returned tuple. This is not the shared-data view introduced for NumPy arrays in Week 2. Here the list structure is deliberately hidden.

4.4 Validate before mutation

In `PracticeHistory.record`, the type and non-empty checks precede `append`. If a check raises an exception, the `append` is never reached. The demo first records a valid label, remembers the history, then tries an invalid label. The unchanged history is the important result, not merely the presence of an error message. This is a **failure-atomic update**: at this public boundary the operation either succeeds or leaves state unchanged. It is not a claim about thread safety or an entire database transaction.

The Assignment applies the same ordering to richer objects. `PlanningRun` owns a `PlanningProblem` and its own initially empty list of `PlanResult` objects. Its `record_result` method must first check that the argument is a result for the same problem. Only then may it append. `results` returns a tuple snapshot, and `latest` returns the last result object, or `None` while history is empty. Copying the small practice class without adding these relationships would not complete TODO 8. The next section defines those relationships.

5 Inheritance and composition express different relationships

5.1 Inheritance is an is-a relationship

A derived class inherits accessible behavior from a base class and may refine it. Week 3 uses one narrow, practical relationship:

Code excerpt - public error type

```
class PlanningModelError(ValueError):
    """A public planning-domain value violated its contract."""
```

A `PlanningModelError` is a `ValueError`. Code that knows the course package can catch the specific type, while a broader caller can still catch `ValueError`.

Inheritance is not used to force unrelated domain nouns into a hierarchy. A path is not a specialized configuration, and a planning problem is not a specialized path. Making those classes inherit from one another would promise substitutability that the domain does not support.

5.2 Composition is a has-a relationship

Composition means that one object holds other objects as parts of its state. A `PlanningProblem` has a start configuration, a goal configuration, and configuration bounds. The following is its field declaration from the Assignment module. It is not a complete standalone class definition:

Code excerpt - `PlanningProblem` fields

```
@dataclass(frozen=True)
class PlanningProblem:
    problem_id: str
    start: Configuration
    goal: Configuration
    bounds: ConfigurationBounds
```

Its post-initialization validates relationships among the parts. The start, goal, and bounds must have the same dimension, and both endpoints must lie inside the bounds. Each component first validates its own local invariant. The composing object then validates the cross-object invariant.

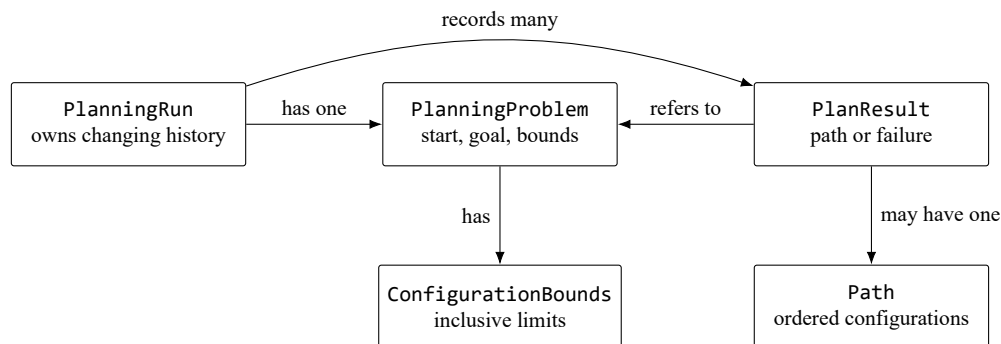


Figure 3.3: Each object validates its local state. Composing objects validate the relationships among those parts.

The diagram is a responsibility map, not a runtime sequence. At runtime a caller constructs objects in dependency order: configurations before bounds and paths, a problem before a result, and a compatible result before recording it in a run.

For example, a lower bound of $(0.0, 0.0)$ and an upper bound of $(0.0, 3.0)$ are individually valid configurations but an invalid pair: the first interval has zero width. `ConfigurationBounds` must check the parts' types, equal dimensions, and strict lower–upper ordering during initialization. Its `contains` method then uses inclusive limits, so a point exactly on an otherwise valid rectangle edge is inside. The strict rule creates a usable region. The

inclusive rule decides membership in that region. For bounds $(0.0, 0.0)$ to $(4.0, 3.0)$, `contains` returns `True` for `Configuration((4.0, 3.0))` and `False` for `Configuration((5.0, 1.0))`. An argument of the wrong type, such as a bare tuple, or the wrong dimension raises `PlanningModelError` instead.

5.3 A result uses cross-field and cross-object invariants

A `PlanResult` contains a problem and either a path or no path. This Assignment excerpt shows its fields and derived property. Validation belongs to the same class's `__post_init__` and is omitted from this excerpt:

Code excerpt - `PlanResult` fields and property

```
@dataclass(frozen=True)
class PlanResult:
    problem: PlanningProblem
    path: Path | None
    message: str = ""

    @property
    def succeeded(self) -> bool:
        return self.path is not None
```

The representation avoids a separate Boolean that could contradict the path. Success is derived from the presence of a path. The constructor then enforces the following rules:

Result form	Required relation	Rejected contradiction
Successful	A path exists. Its dimension and endpoints match. Every waypoint is inside bounds.	path begins elsewhere, ends elsewhere, uses another dimension, or leaves the bounds
Failed	The path is <code>None</code> . A non-empty explanation exists.	no path and no failure explanation
Both	message is a string	numerical or structured message silently accepted as prose

A failed result is still a valid `PlanResult` object: it stores `path=None`, a non-empty explanation, and has `succeeded=False`. A compatible run may record it. In contrast, a constructor exception means no usable result was created because its fields violated the contract. An explained failure and an invalid object are not the same outcome.

This course model does not claim that a path inside rectangular bounds is collision-free. Bounds are the only validity information supplied in Week 3. A later collision checker must add a different contract.

5.4 Collaboration delegates rather than duplicates

The complete caller later in this chapter constructs a supplied path, attaches it to a result, and records that result in a run. Its work passes through several responsibilities:

1. each `Configuration` owns finite coordinates.
2. `Path` owns order and dimensional consistency.
3. `PlanningProblem` owns endpoint and bounds relationships.
4. `PlanResult` connects a path to its problem.
5. `PlanningRun` owns the changing result sequence.

6. `Path.length()` delegates the calculation to the supplied pure function.

No one object is described as a manager of everything. Narrow responsibilities make failures easier to locate and tests easier to state.

6 Tests specify construction and update behavior

6.1 Object tests ask more than whether a method returns a number

Week 2 tests focused on numerical input, output shape, tolerance, and no-mutation behavior. Week 3 retains those ideas and adds object-state questions:

Test category	Question	What the test establishes
Valid construction	Can a coherent object be created?	normalized tuple, dimension, generated representation
Invalid construction	Is an impossible object rejected immediately?	empty configuration, reversed bounds, mismatched endpoints
Ownership	Can a caller mutate internal state through an alias?	source array mutation and returned-array mutation
Valid update	Does one compatible result appear in order?	result count and <code>latest</code> identity
Invalid update	Is failure atomic?	exception plus unchanged history
Instance isolation	Do separate objects own separate mutable defaults?	update one run while the other remains empty
Collaboration	Do composed objects preserve their relationships?	result path matches its problem

6.2 Test the public effect, including the exception type

The small history from Section 4 is already executable, so it is a useful place to verify these ideas before starting the Assignment. Save the following file as `test_practice_history.py` beside `practice_history.py`. The import reads the class from that local file. Each test creates its own objects, so no hidden setup or previously executed cell is required.

Input

Each test constructs its own history and supplies the labels or invalid value shown in its body. No data file or keyboard input is needed.

Code - test_practice_history.py, first two tests

```
import pytest

from practice_history import PracticeHistory

def test_normal_record_keeps_order_and_returns_a_snapshot():
    history = PracticeHistory()
    history.record("first attempt")
    snapshot = history.results
    history.record("second attempt")

    assert snapshot == ("first attempt",)
    assert history.results == ("first attempt", "second attempt")
    assert isinstance(history.results, tuple)

def test_invalid_non_string_label_is_rejected():
    history = PracticeHistory()

    with pytest.raises(ValueError):
        history.record(123)
```

The same file continues with two state-related tests. Neither test reuses an instance from the first two tests.

Code - test_practice_history.py, remaining tests

```
def test_two_instances_have_independent_histories():
    first = PracticeHistory()
    second = PracticeHistory()
    first.record("first attempt")

    assert first.results == ("first attempt",)
    assert second.results == ()

def test_rejected_update_leaves_existing_history_unchanged():
    history = PracticeHistory()
    history.record("valid attempt")
    before = history.results

    with pytest.raises(ValueError):
        history.record(" ")

    assert history.results == before
```

Run the tests from week03_practice. In `python -m pytest`, `-m` runs the named module using the selected Python interpreter. The `-q` option requests a shorter test report. The filename limits this run to the four tests saved beside the practice class:

Run - Anaconda Prompt, week03_practice

```
python -m pytest -q test_practice_history.py
```

Output (summary)

```
4 passed
```

The first test observes both order and an earlier snapshot. The second uses `pytest.raises(ValueError)` because rejecting a number is part of the practice contract. The third establishes separate ownership between instances. The fourth checks an exception *and* unchanged state. Simply seeing an exception would not catch an implementation that appended the invalid value before rejecting it.

These four practice tests are not the eight independent tests required for Assignment 3, Part A. The Assignment tests must call the actual planning package and use independent cases. Its public failure type is `PlanningModelError`, so those rejection tests use `pytest.raises(PlanningModelError)`. That check accepts the named exception type and its subclasses. It does not require an identical error message or expose the internal helper.

The supplied suite also uses parameterization to run the same check with several inputs. An empty coordinate sequence, a Boolean coordinate, and a non-finite coordinate have different reasons for rejection even when they share a test body. Input objects are not automatically copied by parameterization, which is another reason to keep mutations explicit.

6.3 A failure trace identifies the first violated contract

When a test fails, begin with the first failed node ID, then read the exception or assertion, the actual value, and the first line in the editable source. Do not begin by changing several classes at once. The starter first fails in `test_value_objects_reject_field_reassignment` during `Configuration` construction, before its field-reassignment assertion. The selected traceback lines identify the unfinished source operation:

Output excerpt - first failure

```
E   NotImplementedError: TODO 1: validate Configuration
src/ap_week03_planning/domain.py
```

The leading E marks pytest's error explanation, not a command to type. `NotImplementedError` means the intentional TODO is still reached. After an implementation is written, an `AssertionError` instead means a test's asserted relationship was false. For example, a history that changes after a rejected update would fail the fourth practice test's final assertion. Do not interpret both failures as the same diagnosis.

The first dependency is still incomplete. Bounds, paths, problems, and results all depend on valid configurations, so repairing a later class first would not produce a useful checkpoint.

6.4 Worked example: construct, record, and inspect one supplied path

The repository demo uses the same four two-dimensional points introduced in Week 2. It constructs objects. It does not search for the path. The complete caller below can be saved as `worked_objects.py` in the practice folder and run *after* the installed Assignment package has TODO 1–8 completed. Before completion, its first unfinished constructor will raise `NotImplementedError`. It is not another file to submit.

Input

All inputs are supplied in the code. The string `"demo-tool-path"` identifies the problem. The bounds are `(0.0, 0.0)` and `(4.0, 3.0)`. The four ordered coordinates, in metres, are the start, two interior points and the goal. No coordinate entry or input file is needed.

Code - worked_objects.py, construct the problem and path

```
from ap_week03_planning import (
    Configuration, ConfigurationBounds, Path,
    PlanningProblem, PlanResult, PlanningRun,
)

bounds = ConfigurationBounds(
    Configuration((0.0, 0.0)),
    Configuration((4.0, 3.0)),
)
problem = PlanningProblem(
    "demo-tool-path",
    Configuration((0.0, 0.0)),
    Configuration((3.2, 2.0)),
    bounds,
)
path = Path((
    problem.start,
    Configuration((1.0, 1.4)),
    Configuration((2.4, 1.1)),
    problem.goal,
))
```

Append the following lines to the same file. The supplied path becomes a `PlanResult`. The mutable run records that result, after checking that it belongs to this problem. The three `print` calls then read the recorded object rather than a separate list of display values.

Code - worked_objects.py, record and inspect the result

```
run = PlanningRun(problem)
run.record_result(
    PlanResult(problem, path, "validated supplied path")
)
print(f"path length: {run.latest.path.length():.6f}")
print(f"result count: {len(run.results)}")
print(f"succeeded: {run.latest.succeeded}")
```

Run - Anaconda Prompt, week03_practice

```
python worked_objects.py
```

Output

```
path length: 4.356407
result count: 1
succeeded: True
```

These are the three lines produced by the complete caller above. For the fuller supplied demonstration, return to the private repository root and run the protected module. It uses the same inputs but prints three additional descriptive lines:

Run - Anaconda Prompt, repository root

```
python -m ap_week03_planning.demo
```

Output

```
problem_id: demo-tool-path
configuration dimension: 2
waypoints: 4
path length: 4.356407
result count: 1
succeeded: True
```

The calculated length is approximately 4.35640661761539 m. The format `%.6f` displays six decimal places, giving 4.356407. The other lines expose the problem label, dimension, waypoint count, one accepted update and its success property. The demo prints these lines but writes no report files. None of the lines proves that an algorithm discovered the path or that a robot can execute it safely.

6.5 From printed values to inspectable files

The supplied output-generation script writes a JSON report and a browser-readable SVG. The SVG draws the four stored two-dimensional configurations and the rectangular bounds. It helps a human notice a swapped coordinate or missing waypoint, but visual plausibility is not the correctness criterion. Tests establish the public contract. The display makes one executed case inspectable.

Input

The generator uses the same supplied problem, bounds and four coordinates as the demo. It does not read a new input JSON file. Run it only after the package implementation is complete.

Run - Anaconda Prompt, repository root

```
python scripts/generate_outputs.py
```

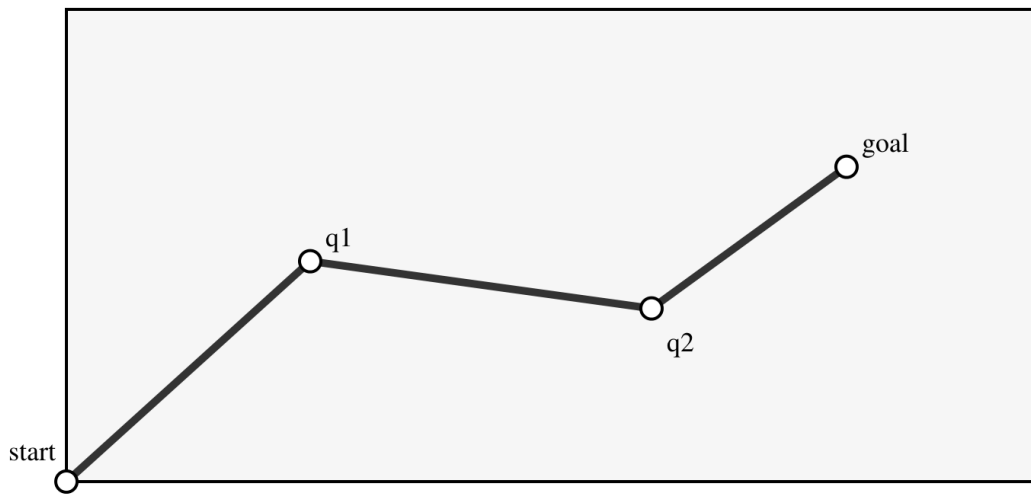
Output

```
[PASS] wrote artifacts\planning_objects_report.json
[PASS] wrote artifacts\planning_objects_preview.svg
[INFO] path_length=4.356407; result_count=1
```

Generated files

- `artifacts/planning_objects_report.json` stores the object values as structured text, including the full computed length.
- `artifacts/planning_objects_preview.svg` displays the same ordered points and bounds in a browser.

Problem: demo-tool-path



Display of supplied path objects; not a planner or physics simulation.

Figure 3.4: The generated SVG rendered from the completed instructor reference. The display represents stored points and bounds. It is not a planner result or a physics-simulation result.

The printed paths identify saved files, not commands to copy into the prompt. In the Assignment’s output-generation step, you will open the SVG, read the JSON and explain how both relate to these output lines. Rerunning the generator replaces these two files from the current object implementation.

Exercise 3.1 Classify a responsibility before writing code

For each behavior, state whether it belongs to a frozen value object, the mutable `PlanningRun`, or the supplied pure geometry function. Justify the owner in one sentence.

1. reject a configuration that contains NaN.
2. add a compatible result to chronological history.
3. sum Euclidean distances between consecutive rows.
4. reject a successful path whose goal differs from the problem goal.
5. give a caller a NumPy representation that it may safely mutate.

Exercise 3.2 Predict the aliasing outcome

Suppose a frozen data class stores a list directly. A caller appends to the original list after construction. Predict the stored field, then rewrite the constructor boundary so the object owns an immutable representation. Explain why `frozen=True` alone did not solve the first version.

Week 4 will begin from the object boundary established here: external scene data must be parsed and validated before it becomes a `PlanningProblem`. The JSON representation may change how data enters the package. It must not bypass the constructor invariants that now protect the domain objects.

7 Assignment 3, Part A: Planning Domain Objects

Part A of Assignment 3 is the Week 3 object-model work described in this chapter. Complete it and submit through LMS within one week of the Week 3 lab, following the same procedure as Weeks 1 and 2. The exact deadline is shown in LMS. Week 4 will extend these objects with collections and a JSON-to-object boundary. That later work does not postpone the Week 3 submission.

7.1 Starting state and required result

The public starter contains seven public object types, two editable source files with TODO 1–8, a published test suite, one placeholder student test, protected callers and scripts, and this Reading. The untouched source raises `NotImplementedError` at the TODO boundaries. This is intentional.

By the end of Part A, the repository must contain:

- completed `domain.py` and `history.py`.
- all 76 published tests passing.
- at least eight independent student tests and at least 84 total passing tests.
- a generated JSON object report and SVG object display.
- a completed engineering note.
- one built wheel.
- one clean private-repository commit with active instructor access, or a correct on-time invitation awaiting only instructor acceptance.
- the three required LMS fields submitted for that Week 3 commit.

The path used by the demo is supplied. A visually reasonable SVG does not replace the tests and does not prove that planning, collision checking, control, or physics simulation occurred.

7.2 Files you edit and files you preserve

Student-editable file	Required work
<code>src/ap_week03_planning/domain.py</code>	TODO 1–7: configuration, bounds, path, problem, and result value objects
<code>src/ap_week03_planning/history.py</code>	TODO 8: the one mutable <code>PlanningRun</code> state owner
<code>tests/test_student_evidence.py</code>	replace the placeholder with at least eight independent tests
<code>artifacts/engineering_note.md</code>	baseline, design explanation, test results, generated-output interpretation, and checkpoint identity

Do not edit the published tests, supplied numerical function, package exports, demo, visualization code, scripts, environment files, build metadata, Reading, README, or protected-file list. Protected scripts generate the JSON, SVG, and wheel. Commit those generated outputs but do not hand-edit them. The grader may restore protected files before checking the editable work.

7.3 Create the private Week 3 repository

After the instructor announces its release, sign in with your existing GitHub account and open the Week 3 course template:

Open in browser - public course template

```
https://github.com/SSUMechE/applied-programming-2026-week03-starter
```

Select **Use this template**, then **Create a new repository**. Use this naming pattern for the private repository:

Repository name - replace YOUR-STUDENT-ID

```
applied-programming-w03-YOUR-STUDENT-ID
```

Under **Settings** → **Collaborators** → **Add people**, invite the exact account SSUMechE. Record whether access is active or the invitation is pending. You may clone and work while instructor acceptance is pending. Accepting the invitation gives the instructor access. Your own clone authentication is a separate step.

Open **Anaconda Prompt**, not Python's >>> prompt. Change to the parent folder used for course repositories. Replace the example folder and both placeholders below with your own values. The two `set` commands store short names so the clone command is easier to enter:

Run - Anaconda Prompt, start in your course parent folder

```
cd /d "C:\your-course-folder"
set "OWNER=YOUR-GITHUB-ID"
set "REPO=applied-programming-w03-YOUR-STUDENT-ID"
git clone https://github.com/%OWNER%/%REPO%.git
cd applied-programming-w03-YOUR-STUDENT-ID
git status
git remote -v
```

Here `git clone` downloads that repository into a new local folder. `git status` reports its branch and working-file state, while `git remote -v` shows the saved remote names and addresses. Both `origin` lines must name the student's private repository. Do not submit work whose origin still names the public template. If you already cloned your private repository, reopen that folder instead of creating another copy. These `set` and `cd /d` forms are Windows Command Prompt syntax. On other systems, use the terminal commands from Week 2 and clone the HTTPS URL copied from your private repository's **Code** menu.

For the first HTTPS clone, Git Credential Manager may open a browser. Sign in as the private repository's owner. If Git asks for a password instead, an ordinary GitHub account password is not accepted for Git operations. Use the Week 1–2 authentication guidance for a credential manager or a personal access token with access to this repository. Never put a token in the clone URL, code, engineering note, screenshot, or LMS entry. **Repository not found** may mean the wrong URL or signed-in account, not a Python installation problem.

Procedure 3.1 Install and verify the declared environment

Run every command from the repository root. Reuse the Week 1 course environment. Activation selects Python, installing the requirements supplies the external tools, and editable installation makes this repository's own package importable:

Run - Anaconda Prompt, repository root

```
conda activate applied-programming-2026
python -m pip install -r requirements.txt
python -m pip install -e . --no-build-isolation
python scripts/verify_environment.py
```

Output (summary)

```
[PASS] Week 3 object-programming environment is ready.
```

This status belongs to the verifier, not the pip installation commands. The verifier checks the interpreter, pinned packages, import location, and public names without executing any TODO body. A correct environment may therefore pass while the unfinished Assignment still fails its tests.

The `-r requirements.txt` option tells pip to read the required package versions from that file. In `-e .`, the dot means the current repository directory and `-e` requests an editable installation. Imports then use the source being edited here, so rerunning Python sees saved TODO changes without rebuilding a wheel each time. This does not implement the TODOs.

`--no-build-isolation` uses the build tools already installed in the active environment instead of creating a separate build environment. That is why the requirements command comes first. The final command runs the supplied verifier file. If installation fails, resolve that error before continuing to the verifier or baseline.

Only if activation reports that the named environment does not exist, run the following two commands, then repeat the two pip commands and verifier above:

Run - Anaconda Prompt, repository root

```
conda env create -f environment.yml
conda activate applied-programming-2026
```

Do not delete an existing course environment. If `conda` is not found, first make sure you opened Anaconda Prompt. The supported Windows alternative when Anaconda is unavailable is an existing Python 3.12 installation. The first command creates a local `.venv` directory and the second activates it for this prompt. Use this alternative instead of Conda creation, not in addition to a working course environment:

Run - Windows Command Prompt, repository root (alternative)

```
py -3.12 -m venv .venv
.venv\Scripts\activate
```

Then run the same two pip commands and verifier from the repository root.

Procedure 3.2 Record the intentional baseline before editing**Run - Anaconda Prompt, repository root**

```
python scripts/run_baseline.py
```

Output (summary)

```
68 failed, 8 passed
```

The inner pytest run fails intentionally. The wrapper `run_baseline.py` succeeds only when it recognizes the

expected 68 failed, 8 passed summary and prints its own PASS line. That PASS confirms the starting state, not a passing test suite. Next open the existing note. The following command opens an editor, not another test:

Run - Anaconda Prompt, repository root

```
notepad artifacts\engineering_note.md
```

In section 0 of the existing note, record the summary and the first failed pytest node ID. Save with **Ctrl+S**. Do not overwrite your answers with a new template. If collection fails or a different count appears, stop. Do not edit published tests to manufacture the expected count.

```
self = Configuration(values=(1.0, 2.0))

def __post_init__(self) -> None:
    # TODO 1: normalize values inside this frozen dataclass.
>     raise NotImplementedError("TODO 1: validate Configuration")
E     NotImplementedError: TODO 1: validate Configuration

src\ap_week03_planning\domain.py:35: NotImplementedError
```

Figure 3.5: Actual first-failure output from the untouched starter, cropped from the VS Code Debug Console. Pytest includes the source lines that raised `NotImplementedError` and identifies `domain.py:35`. This is an unfinished TODO, not a failed numerical assertion. The instructor ran `python -m pytest -q tests/test_published_contract.py -x`. The run summary, outside this excerpt, was 1 failed, 7 passed. Stopping at the first failure is different from the full 68 failed, 8 passed baseline above.

7.4 Complete TODO 1–8 in dependency order

The table below states the exact student work and the first useful checkpoint. Later objects depend on earlier ones, so complete the rows in order.

TODO	Object or method	Completion condition
1	<code>_normalized_values</code> <code>Configuration.__post_init__</code>	Validate a non-empty 1-D real finite sequence. Reject Boolean, complex, object-dtype, string, non-finite and wrong-shaped input. Store owned floats.
2	<code>Configuration.dimension</code> <code>Configuration.as_array</code>	return the tuple length and a new <code>float64</code> array that cannot mutate the object
3	<code>ConfigurationBounds.__post_init__</code> <code>ConfigurationBounds.dimension</code>	require two configuration objects of equal dimension and strict <code>lower < upper</code> in every coordinate
4	<code>ConfigurationBounds.contains</code>	Return <code>True</code> inside or on limits and <code>False</code> outside. Reject incompatible type or dimension.
5	Path methods and properties	own a tuple of at least two configurations of one dimension. Expose <code>start</code> , <code>goal</code> , <code>array</code> and delegated <code>length</code> .
6	<code>PlanningProblem.__post_init__</code>	Strip ID whitespace and reject an empty ID. Require typed, compatible parts and endpoints inside bounds.

TODO	Object or method	Completion condition
7	<code>PlanResult</code>	represent a matching successful path or an explained failure. Derive <code>succeeded</code> . Reject contradictory combinations.
8	<code>PlanningRun</code>	Create one list per instance. Validate before append. Return a history snapshot and latest result without exposing the list.

Every rejected public input must raise `PlanningModelError`, including array-conversion failures. Check mixed Boolean inputs before numeric coercion. `Path` accepts a sequence of configuration objects and owns a tuple of them. `PlanResult` checks that the path stays within its problem's bounds and matches its endpoints. Those checks do not imply collision checking. `PlanResult.message` must be a string and is stored with leading and trailing whitespace removed. Failure requires a non-empty message after that operation. Success may use an empty message. An empty run has `latest` is `None`. After recording, `latest` is the actual last result object, not a newly reconstructed copy.

Procedure 3.3 Checkpoint configurations and bounds

After TODO 1–4, run:

Run - Anaconda Prompt, repository root

```
python -m pytest -q tests/test_published_contract.py -k test_configuration_ -x
```

Output (summary)

```
21 passed, 55 deselected
```

Run - Anaconda Prompt, repository root

```
python -m pytest -q tests/test_published_contract.py -k test_bounds_ -x
```

Output (summary)

```
11 passed, 65 deselected
```

These are two separate runs. Their counts are not added together as a full-suite result.

If the first command stops at `object dtype`, Boolean, complex, or non-finite input, read which clause of the contract is still missing. Do not silently convert a string or object array to make the test pass. The `-k` option selects matching test names, and `-x` stops after the first failure. Here the specific name prefixes avoid selecting later classes' tests. `deselected` counts tests intentionally left out of this command, not additional failures. A focused pass is a local checkpoint, not full completion.

Procedure 3.4 Checkpoint path and problem composition

After TODO 5–6, run:

Run - Anaconda Prompt, repository root

```
python -m pytest -q tests/test_published_contract.py -k test_path_ -x
```

Output (summary)

```
8 passed, 68 deselected
```

Run - Anaconda Prompt, repository root

```
python -m pytest -q tests/test_published_contract.py -k test_problem_ -x
```

Output (summary)

```
9 passed, 67 deselected
```

The path test includes ownership, start and goal, array shape, dimension, and length. The problem test adds relations among composed objects. Passing one command does not imply that later object relations are complete.

Procedure 3.5 Checkpoint result and state ownership

After TODO 7–8, run:

Run - Anaconda Prompt, repository root

```
python -m pytest -q tests/test_published_contract.py -k "result or run" -x
```

Output (summary)

```
16 passed, 60 deselected
```

The run tests include two-instance isolation and rejected updates. Validate a candidate result before appending it so a raised exception leaves history unchanged.

Now remove the name filter and run the entire published test file:

Run - Anaconda Prompt, repository root

```
python -m pytest -q tests/test_published_contract.py
```

Output (summary)

```
76 passed
```

This second command checks all published cases, including the earlier objects. Only this full-file result is the published-suite completion checkpoint.

7.5 Add eight independent tests

Replace `test_student_placeholder` with at least eight top-level `test_...` functions. Do not copy or merely rename published cases. Use new numerical values and include all of the following cases:

1. one valid configuration normalization case.
2. one invalid representation or non-finite case.
3. one inclusive bounds case.
4. one aliasing or returned-copy case.
5. one path ownership or length case.
6. one invalid composed problem or result case.
7. two independent run histories.
8. an invalid state update that raises and leaves history unchanged.

Run the entire visible suite:

Run - Anaconda Prompt, repository root

```
python -m pytest -q
```

Output (summary)

```
84 passed
```

The shown summary assumes exactly eight independent student tests. At least 84 tests must pass: 76 published cases plus at least eight independent student cases. Skipped tests do not count as passing cases. A larger number is not automatically better. Each test must make a distinct claim that its name and assertions reveal.

Procedure 3.6 Generate and interpret the object report and display

The demo and generator have different outputs. First inspect the six printed values. The actual completed-reference capture below shows the protected demonstration from Section 6.4. The untouched starter will not produce this successful result before its TODOs are implemented.

Run - Anaconda Prompt, repository root

```
python -m ap_week03_planning.demo
```

Output - actual instructor-reference capture

```
problem_id: demo-tool-path
configuration dimension: 2
waypoints: 4
path length: 4.356407
result count: 1
succeeded: True
```

Now generate the saved files from the same fixed inputs:

Run - Anaconda Prompt, repository root

```
python scripts/generate_outputs.py
```

Output

```
[PASS] wrote artifacts\planning_objects_report.json
[PASS] wrote artifacts\planning_objects_preview.svg
[INFO] path_length=4.356407; result_count=1
```

The demo reads its fixed, supplied coordinates and prints six summary lines. It writes no files. The second command constructs the same objects through the protected caller and writes the following two files, then prints their paths. It needs no additional input file:

Generated files

- artifacts/planning_objects_report.json: the object-value report.
- artifacts/planning_objects_preview.svg: the browser-readable display.

In Anaconda Prompt, open the SVG in its associated browser and read the JSON:

Run - Anaconda Prompt, repository root

```
start "" artifacts\planning_objects_preview.svg  
type artifacts\planning_objects_report.json
```

If the SVG opens in an editor instead, use the browser's Open File command. The `start` command opens the file with its associated application. Its empty quotes provide an empty window-title argument. The `type` command prints the saved JSON text without editing it. Confirm that the display contains four ordered points, the stated start and goal, and the rectangular bounds. Then inspect the JSON and confirm `problem_id`, dimension, endpoints, waypoint count, length, result count, and success state. Record what each file establishes and what it does not establish in the engineering note. The JSON retains the computed floating-point length rather than the demo's six-decimal formatting. Rerunning the generator replaces these two generated files from the current implementation. It does not overwrite the engineering note or implement any TODO.

The completed-reference capture is cropped from the VS Code Debug Console. The same module prints these six lines in Anaconda Prompt. It does not replace tests, the engineering note, package checks or LMS submission.

Procedure 3.7 Complete the engineering note

Run - Anaconda Prompt, repository root

```
notepad artifacts\engineering_note.md
```

This opens the existing note for editing. Replace every `REPLACE_ME`, using your own observed results and explanations. Do not merely create an empty Markdown file. The note must not describe the SVG as a planned, collision-free, physically executed, or safe robot motion. It must explain:

- the intentional baseline and first failed test.
- value-object responsibilities and the one mutable owner.
- one constructor invariant and one atomic update invariant, with the student's independent test names.
- the error inheritance relationship and one domain composition relationship.
- aliasing protection in configuration, path, and history.
- how Week 4 can create these objects from JSON without weakening their invariants.

Save with **Ctrl+S**, close the editor, and verify the saved file:

Run - Anaconda Prompt, repository root

```
type artifacts\engineering_note.md
```

Read each answer again. No placeholder or empty response should remain. Keep the final commit ID outside the note: editing a committed note to insert that commit ID would require a new commit with a different ID.

Procedure 3.8 Build and check the package

The editable installation used during development points to the current source. A **wheel** is a separate `.whl` package file that can be installed without that editable link. Building it checks that the project can produce the distributable required by this checkpoint. From the repository root, inspect `dist`. If an earlier Week 3 wheel exists, remove only that old `.whl` file so the final folder contains one wheel. Then run:

Run - Anaconda Prompt, repository root

```
python -m build --wheel --no-isolation
dir /b dist\*.whl
```

Output from dir /b - wheel filename

```
ap_week03_planning-0.1.0-py3-none-any.whl
```

--wheel selects wheel output. --no-isolation makes the build use the active environment's installed build tools. It does not disable tests and does not change the later checker's isolated-interpreter import check. The dir /b command lists matching filenames without extra directory details. For the unchanged package metadata, its listed filename is shown above.

This file is inside dist. Finding the file confirms a build output, not a completed Assignment. Run the supplied checker next:

Run - Anaconda Prompt, repository root

```
python scripts/check_submission.py
```

The checker requires the public and student tests, at least eight top-level student test functions, both generated output files, a completed engineering note, and exactly one ap_week03_planning wheel that passes an isolated import smoke test. That check runs a separate isolated Python process and checks that the course package is imported from this wheel rather than the editable source. It uses existing dependencies, not a newly installed environment. Saved generated outputs are compared with a fresh generation. The script can verify test counts and execution, but the grader must inspect whether the claims and explanations are correct and genuinely independent. A PASS line confirms only these visible checkpoint conditions. The instructor suite may exercise additional values and object relationships.

Procedure 3.9 Commit and push an identifiable checkpoint

Inspect the source state before committing:

Run - Anaconda Prompt, repository root

```
git status --short
git diff -- src/ap_week03_planning/domain.py
git diff -- src/ap_week03_planning/history.py
git diff -- tests/test_student_evidence.py
```

status --short lists changed or untracked paths. Each diff shows the unstaged edits in the named file, so check that the changes are your intended implementations and tests. If a diff shows a pager, press q to return to the prompt.

Stage the allowed source, independent tests, note, generated JSON and SVG files, and wheel. Commit and push:

Run - Anaconda Prompt, repository root

```
git add src/ap_week03_planning/domain.py
git add src/ap_week03_planning/history.py
git add tests/test_student_evidence.py artifacts dist
git diff --staged
```

After checking the staged changes, record and upload the checkpoint:

Run - Anaconda Prompt, repository root

```
git commit -m "Complete Week 3 planning domain objects"
git push origin main
git rev-parse HEAD
git ls-remote origin refs/heads/main
git status --short
```

`git add` selects saved changes for the next commit. It does not upload them. `git diff --staged` shows the selected changes, and `git commit` records them locally. `git push origin main` uploads the branch to the saved remote. `rev-parse HEAD` prints the local commit ID, while `ls-remote` reads the remote branch ID for comparison. Neither inspection command creates another commit.

The final status must be empty. The printed commit identifier must contain 40 hexadecimal characters, and the same commit must be visible on the private GitHub repository. The two full IDs printed by `rev-parse` and `ls-remote` must agree. Confirm again whether SSUMechE has active access or a correctly addressed pending invitation. Sending the invite does not itself grant access, but instructor acceptance is not an action that the student can complete.

7.6 Submit the Week 3 work through LMS

Use the same three LMS fields as Weeks 1 and 2:

1. your private `applied-programming-w03-<student-id>` repository URL.
2. the full 40-character commit ID printed by `git rev-parse HEAD`.
3. confirmation that the repository is private, the actual SSUMechE access status, and that this commit is to be graded.

Submit within one week of the Week 3 lab. Follow the exact date and time in LMS. Do not wait for Week 4. The code, note, generated outputs, and wheel belong in the repository. No separate LMS ZIP or screenshot attachment is required.

The following is a fictional three-field example. Replace its identity and hash:

LMS entry - fictional format example

```
Repository URL:
https://github.com/student-example/applied-programming-w03-20261234
Full commit ID:
0123456789abcdef0123456789abcdef01234567
Confirmation:
The repository is private. SSUMechE has active collaborator access.
The commit above is the revision to be graded.
```

Do not submit the public template URL, a local folder path, a branch name or `latest`, a shortened hash, or a commit that has not been pushed. Never include passwords, tokens, or other credentials.

If the correct invitation was sent on time and only instructor acceptance is pending, report that actual state instead of claiming active access. This is not a student omission. A missing or misaddressed invitation must be corrected.

GitHub upload alone is not a submission. Missing LMS submission, an incorrect repository name or required file structure, or unmet mandatory submission requirements by the deadline results in zero for this submission. A failed numerical test is assessed under the technical criteria, not automatically treated as a missing submission. Later pushes do not replace the commit ID already submitted through LMS.